ENGLISH EDITION

Automatic Pond Aeration

The complete user manual

Control and monitor a pond aeration system with ioBroker — from your very first step.

WARNING **WARNING — please read before you start**

This adapter is **still under development** and **not yet verified for unattended use**. It controls a **life-support system for living animals**. A malfunction, a wrong setting or a bug can stop the aeration and **endanger the health and life of your fish** (oxygen depletion, no ice-free hole in winter, an overheating pump). **Do not rely on it unchecked:** watch it closely, verify every function on your own hardware, and keep an independent, proven aeration/failsafe in place. **Use at your own risk.**

ADAPTER
ioBroker.automatic-pond-aeration

VERSION
v0.0.14

DATE
8 July 2026

Contents

1 Welcome

This manual explains the **ioBroker.automatic-pond-aeration** adapter from the ground up. It is written for readers with **no prior knowledge** — if you have never used ioBroker and have never wired a valve, you are in the right place. By the end you will be able to install the adapter, connect it to your aeration hardware, configure it safely, and operate it from your phone or computer.

1.1 What this adapter does

A pond aeration pushes air from a pump through hoses to **air stones (diffusers)** on the pond floor. The rising bubbles add oxygen and keep the water moving. This adapter decides **when** and **where** the air flows, by opening and closing **valves** — one valve per *aeration point* (up to 8). It can:

- switch the aeration points by a **weekly schedule**, a **cyclic sequence** (round-robin over points and/or groups), or in **groups**;
- protect the pump with a **safety interlock** so it never runs against closed valves;
- optionally **monitor** dissolved oxygen, air/water temperature and air-line pressure, and raise alarms;
- keep an **ice-free hole** in winter and **boost aeration when oxygen is low**;
- **pause** the aeration while your fish are being fed (coupling to the automatic-feeder adapter);
- send **notifications** (e.g. Telegram) and collect **runtime statistics**.

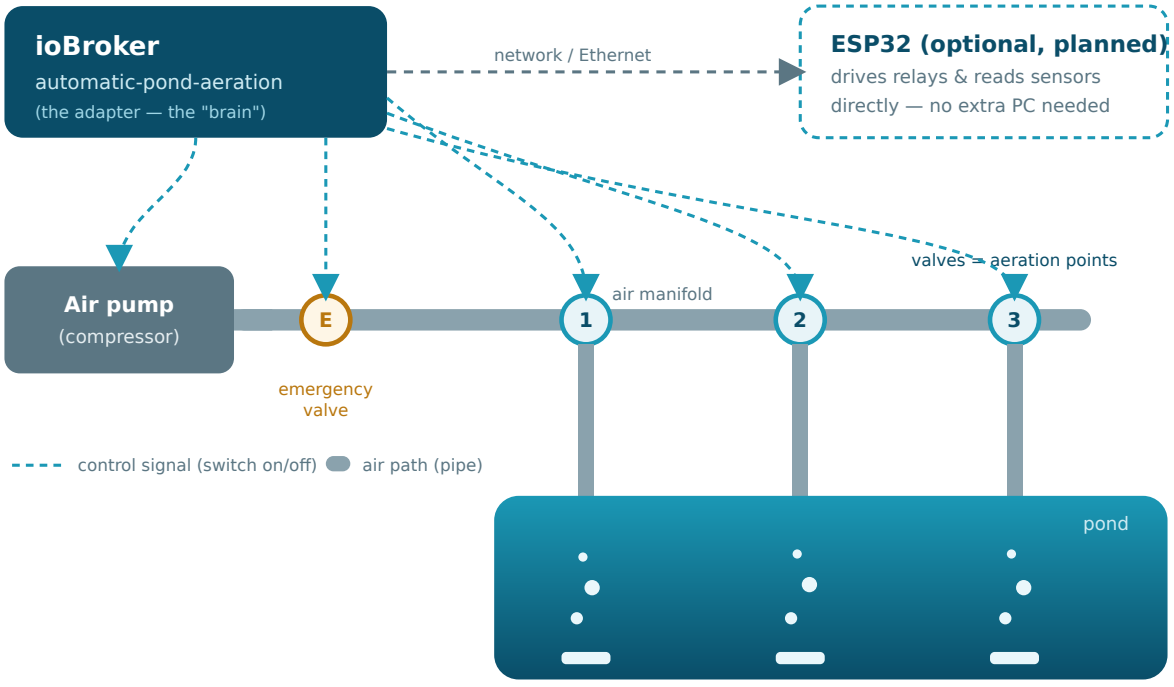

Figure 1: How the parts fit together: the adapter (the “brain”) decides which valves open; the pump feeds air through the manifold; each open valve sends air to one diffuser in the pond. The adapter can alternatively drive the hardware directly via an ESP32 [3].

1.2 Who should read which part

1 **Just want to understand it?** Read chapters 1–3.

2. **Setting it up?** Follow chapters 4–6 in order.
3. **Operating it day to day?** Chapter 7 and the FAQ (chapter 10).
4. **Something not working?** Jump to Troubleshooting (chapter 11).

1.3 Quick start (the short version)

> **NOTE** **If you already run ioBroker and have a switchable valve**
>
> This is the fastest path. Every step is explained in detail later — the chapter is named in brackets.

1. Install the adapter and add an instance (chapter 5).
2. Turn on **Dry-run** so nothing is switched yet (General tab).
3. Add your **aeration points** and map each one to its valve state (chapter 6).
4. Set a simple **schedule** or enable the **round-robin** so something happens (Control tab).
5. Watch the log and the `aeration.point.*.active` states — confirm the right points turn on/off.
6. Configure **Safety** (pump + emergency valve), then turn **Dry-run off** and supervise the first real runs closely.

> **SAFETY** **Do not skip the supervised phase**
>
> Because the animals depend on the aeration, run it **watched** for a while before you trust it unattended — see the warning on the cover.

2

How a pond aeration works (the basics)

Even if this is all new to you, the idea is simple.

2.1 The physical parts

Air pump / compressor	Produces the airflow. Often a linear-diaphragm pump. Runs at low pressure (typically a few kPa up to ≈30 kPa).
Manifold + hoses	Distribute the air to several points in the pond.
Valves (solenoids)	One per aeration point. Open = air flows to that diffuser; closed = it doesn't. These are what the adapter switches.
Diffusers / air stones	Turn the airflow into fine bubbles under water.
Emergency valve	A safety relief so the pump always has somewhere to blow, even if every normal valve is closed.

2.2 Why aeration matters for the animals

Fish and pond life need **dissolved oxygen** in the water. Warm summer nights, algae and decaying matter can deplete it; a thick ice sheet in winter traps toxic gases. Aeration adds oxygen, mixes the water, and — in winter — keeps a small **ice-free hole** open so gases can escape [2]. Because the animals depend on it, **reliability and safety come first** — which is why this adapter is built around a hard safety interlock (chapter 3).

3

Safety concept (the most important chapter)

An air pump must **never run against fully closed valves** (“dead-heading”): the pressure has nowhere to go, the pump overheats and can be damaged. The adapter prevents this at all times.

SAFETY **The dead-head interlock**

While the pump runs, at least one valve is **always kept open** (you set the minimum). If that cannot be guaranteed, the adapter **opens the emergency valve** and — if the pump is controllable — **switches the pump off**.

Two more safety mechanisms work alongside it:

- **Make-before-break switching:** when the aeration moves from one point to the next, the new valve opens **before** the old one closes, so there is never a moment with everything shut.
- **Watchdog:** the interlock is re-checked on a short timer, not only when something changes.

TIP **Wiring recommendation**

Use a **normally-open (NO)** emergency valve, so it opens by itself on a power cut — a true failsafe. When you later run the hardware on an ESP32, the same interlock also runs **on the device**, so a network or ioBroker outage cannot harm the pump [3].

4 What you need

An ioBroker installation	The smart-home server this adapter runs in [1]. Version: js-controller ≥ 6.0.11, admin ≥ 7.6.20.
Node.js ≥ 22	Required by the adapter.
At least one switchable valve	Reachable in ioBroker as a **state** — e.g. from a relay board, smart plug or KNX/Zigbee actuator. The adapter switches these existing states.
Optional	A controllable pump, an emergency valve, and sensors for oxygen / temperature / pressure — see chapter 9.

NOTE **No ioBroker yet?**

Install ioBroker first (see the official documentation [1]). It runs on a Raspberry Pi, a NAS, a small PC or a virtual machine. Once the ioBroker admin opens in your browser, come back here.

5

Installing the adapter

1. Open the **ioBroker admin** in your browser (usually `http://<your-server>:8081`).
2. Go to **Adapters**, click the **GitHub / custom install** icon (the cat/octocat), and enter the repository URL `https://github.com/ssbingo/ioBroker.automatic-pond-aeration` [2]. (Once the adapter is in the official repository you will simply search for “pond aeration”.)
3. Confirm the install and wait until it finishes.
4. Click **Add instance**. A new instance `automatic-pond-aeration.0` is created and its settings page opens.

OK Done

You now have an instance. Nothing is controlled yet — the **master switch** is on, but you have not told it about any valves. That is the next chapter.

6

Configuring the adapter, tab by tab

The settings page is organised into tabs. You only fill in the parts you use. Click **Save** (or **Save and close**) when you are done.

6.1 General

- **Master enable** — the on/off switch for the whole adapter. Off = nothing is controlled.
- **Dry-run (log only)** — a **test mode**: the whole logic runs and the data points update, but no real valve or pump is switched — the intended actions are only written to the log (`[DRY-RUN] would …`). Perfect for trying a configuration safely before wiring it up.
- **Backend** — `Existing ioBroker states` (default) drives your hardware through other adapters' states. `ESP32 (direct)` talks to the separate reference firmware on a Waveshare board over HTTP (set the host/IP and the emergency-valve / pump relay channels; see chapter 9). The firmware is still being completed.

6.2 Aeration points

This is the heart of the setup. Add **up to 8** points; each one is one valve.

1. Click **Add point**.
2. Give it a **name** (e.g. "Pier", "Deep zone").
3. Leave **Backend** on `ioBroker`.
4. Under **Valve state**, pick the ioBroker state that opens this valve (the object browser lets you search your existing states).

> **SAFETY** **Each enabled point needs a valve state**
>
> If a point is enabled but has no state mapped, the adapter warns in the log and cannot switch it.

Each point may also have an optional **override button** — a physical push-button (e.g. an ESP32 digital input, or any boolean state). It works as a **toggle**: one press forces that point **on with priority over the automatic control** (schedule, sequence, winter, oxygen) and even over a feeder pause; only the master switch or a safety trip overrides it. Press again to release. A button is only available for an **aeration valve**: a point that sits on the ESP32 pump or emergency-valve relay channel cannot have one (the option is greyed out, because those channels are safety-critical). With the ESP32 backend, a button pressed **at the device** is reflected back into ioBroker (`aeration.point.<n>.buttonOn`) and gets the same priority. (More button modes are planned.)

6.3 Groups

Group several points to control them together (e.g. one switch opens three diffusers). Give the group a name and tick its member points.

> **NOTE** **Hard rule**
>
> There can **never be more groups than points**. The admin and the adapter both enforce this.

6.4 Control

Here you decide **when** the aeration runs automatically.

- **Cyclic round-robin** — rotate through the points, each open for the **dwell time** (seconds).
  - **Sequence (points and groups)** — optionally define an **ordered cycle of steps**, where each step targets a single point **or** a whole group, with its own optional dwell time. This lets you run e.g. *group 1 → group 3 → point 1 → …* and freely mix points and groups. Reorder steps with the up/down arrows. An empty sequence just rotates through all points.
- **Schedules** — open selected points/groups during weekday time windows. Pick **From** and **To** from a **clock picker** (hour/minute, 24 h; overnight windows like `22:00–06:00` work too). **An active schedule has priority over the round-robin / sequence.**
- **Winter / ice-free mode** — pick the season **start** and **end** from a **calendar** — only the **day and month** count and the window recurs every year (e.g. 1 Nov – 15 Mar, which correctly wraps across New Year). The chosen points are then forced on to keep an ice-free hole open. Optionally tick **“only when it is cold”** and set an air-temperature threshold so the pond is only aerated while it is actually freezing (needs air-temperature monitoring). Leave the point selection empty to aerate the whole pond.

6.5 Sensors

Optional monitoring. For each sensor tick **Enabled** and pick the **source state**.

- **Dissolved oxygen** — with a low threshold (raises an alarm), a target and a hysteresis. The oxygen **saturation %** is computed from the water temperature.
  - **Oxygen closed loop** — when enabled, the adapter **forces aeration on** while the oxygen is below the low threshold, until it recovers to the target. A safety net for the fish.
- **Air / water temperature.**
- **Pressure** — with a min/max range; leaving the range raises a pressure alarm. Useful to spot a blocked diffuser (pressure rises) or a burst hose (pressure drops).

6.6 Location

Needed only for the astronomical times (sunrise/sunset/night). Choose the ioBroker system location, or enter a custom address — the map geocodes it via OpenStreetMap/Nominatim on demand [15].

6.7 Feeder

Pause selected aeration points while the *automatic-feeder* adapter [13] is feeding, so the food is not blown around. Pick the feeder instance (auto-discovered) and the switches to watch, choose a **duration mode** (measure/pulse) and an **offset** (extra pause after feeding — at least the average time the animals need to eat).

6.8 Safety

- **Min. open valves while the pump runs** — the dead-head protection (default 1).
- **Watchdog interval** and **make-before-break overlap**.
- **Pump** — whether it is controllable (then the interlock may switch it off), its state, and anti short-cycle on/off times.
- **Emergency valve** — its state, whether it is **normally open** (failsafe), the valve **type** (solenoid or motorised ball valve) and, for a motor valve, its travel time.

6.9 Notifications

Enable notifications and pick a **messaging instance** (any adapter of type `messaging`, e.g. Telegram or Pushover), then **tick which events** should send a message:

- **safety interlock** — when it trips or clears;
- **oxygen alarm** — when dissolved oxygen drops too low or recovers;

- **pressure alarm** — when the pressure leaves or re-enters its range.

A short, localized text is sent on each edge. With no event ticked, nothing is sent.

> **TIP** **Feeding does not spam the interlock notification**
>
> When the feeder pauses the aeration points, all valves close and the emergency valve opens — that is normal and expected, so the **interlock notification is suppressed for the duration of the feeding pause**. A genuine problem still reaches you: if the pump really dead-heads against closed valves the **pressure alarm** fires and is reported on its own. An interlock trip that **persists after feeding ends** is escalated as usual. If you nevertheless want to be alerted for the interlock during feeding, enable **Also notify the interlock during feeding** on the Notifications tab.

7 Using the adapter day to day

The adapter exposes **data points** (states) you can read and command — from the ioBroker admin, from scripts, or from a visualization. The most important commands:

`control.enabled`	Master on/off.
`control.mode`	`auto` (schedules/sequence/winter/oxygen run automatically), `manual` (you open points by hand), or `off` (all closed).
`control.allOff`	Close every valve immediately.
`control.point.<n>.open`	Open/close one point by hand (only takes effect in `manual` mode).
`control.group.<g>.active`	Activate a group.

TIP **How the modes interact**

In `auto` mode the automatic programs (schedule, sequence, winter, oxygen boost) decide the valves; the safety interlock always runs on top, and a feeder pause always wins. In `manual` mode you are in control, but safety and the feeder pause still apply.

8 Data points reference

The adapter creates these states from your configuration. `<n>` = point index (0–7), `<g>` = group index. Items marked **(w)** are writable commands; the rest are read-only status values.

Object	Meaning
`info.connection`	Adapter running / configuration valid
`info.activeMode`	Current operating mode
`info.dryRun`	Dry-run active (no hardware switched)
`info.deviceFirmware` / `.firmwareCompatible`	ESP32 firmware version / protocol-compatible flag (ESP32 backend)
`info.licenseTier` / `.licenseTrialDaysLeft`	ESP32 licence tier (free/community/pro) / trial days left
`info.deviceCode` / `.licenseControlBlocked`	ESP32 device code for unlocking / control rejected (not licensed)
`control.enabled` **(w)**	Master enable
`control.mode` **(w)**	`auto` / `manual` / `off`
`aeration.point.<n>.valveState`	Valve is open
`aeration.point.<n>.active`	Point is currently aerating
`aeration.point.<n>.buttonOn`	Override button active (only with a button configured)
`aeration.point.<n>.runtimeTodaySec` / `.runtimeTotalH`	Runtime today / total
`safety.interlockActive`	Safety interlock currently active
`safety.emergencyValve`	Emergency valve is open
`safety.openValveCount`	Number of open valves
`sensors.oxygen` / `.oxygenSaturation` / `.oxygenAlarm`	Oxygen value / saturation % / low alarm
`sensors.oxygenBoostActive`	Oxygen closed loop forcing aeration on
`sensors.pressure` / `.pressureAlarm`	Pressure value / out-of-range alarm
`winter.active` / `.frostActive`	Winter mode forcing on / frost protection engaged
`statistics.compressorRuntimeTodayH`	Compressor runtime today
`statistics.switchCyclesToday`	Valve switch cycles today

9 Hardware & wiring (reference build)

The adapter drives your valves and pump either through **existing ioBroker states** (any relay board or smart plug works) or **directly via an ESP32**, so you need no extra PC — this section is the reference for the ESP32 build.

9.1 Controller board

The reference controller is the **Waveshare ESP32-S3-POE-ETH-8DI-8RO** [3]: 8 relays (for the valves, emergency valve and pump), 8 digital inputs, and Ethernet with **Power-over-Ethernet** — one cable for power and data.

> **NOTE** **You do not need this board today**
>
> With the current ioBroker backend, **any** relay board or smart plug that exposes each valve as a state works. The diagrams below apply equally: “relay board” is whatever hardware switches your valves.

9.2 Failsafe wiring (very important)

Wire the actuators so that **losing power leaves the pond safe**. A relay that loses power **releases** (de-energises), so choose each device’s wiring accordingly:

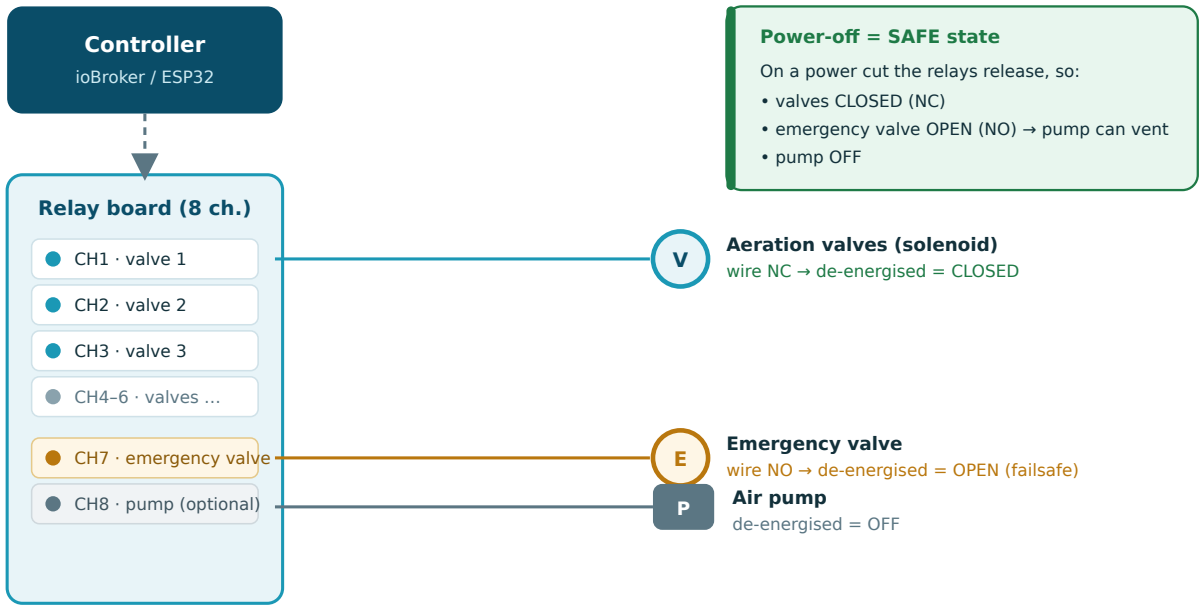

Figure 2: Failsafe wiring: aeration valves as **normally-closed (NC)**, the emergency valve as **normally-open (NO)**, the pump switched off when de-energised. On a power cut everything falls into the safe state by itself.

Aeration valves	Wire **normally-closed (NC)**: no power → closed. The adapter energises a relay to **open** a point.
Emergency valve	Wire **normally-open (NO)**: no power → open, so the pump always has somewhere to blow. This is the real failsafe.

Pump	If controllable, wire so de-energised = off. If you only **observe** the pump, the interlock still opens the emergency valve.

> **SAFETY** **Motorised ball valves do not spring open**
>
> A motor ball valve (e.g. CWX-15N) **keeps its position** on power loss — it is not fail-open. If you use one as the emergency valve, the dead-head protection then relies on the pump also losing power. For a true failsafe, prefer a **normally-open solenoid** emergency valve.

9.3 Reference sensors (all optional)

Water temperature	**DS18B20** waterproof probe (1-Wire). ≈€3 from a reputable shop [16]. Buy from an authorised distributor — most cheap clones are counterfeit [12].
Air-line pressure	**CFSensor XGZP6897D…KPDG** (I²C), gauge type, 0–100 kPa default [7]. Only available from AliExpress/Alibaba; the firmware auto-detects the two bus variants (0x6D / 0x58).
Dissolved oxygen	**Optional and costly.** Budget: **DFRobot SEN0237-A** (≈ €176) [6]. Premium: **Atlas EZO-DO** stack (≈ €450) [4], which needs an electrical isolation carrier [5].

Wiring and the I²C address map are covered in the **Wiring the sensors to the ESP32** section below.

> **SAFETY** **Oxygen sensing is maintenance-heavy**
>
> Both oxygen options use a galvanic probe with a membrane/electrolyte that must be serviced regularly and needs water flow across it. Treat oxygen as an optional, advanced add-on.

9.4 Wiring the sensors to the ESP32

All three reference sensors run at **3.3 V**, so there is **no level shifting**. The two I²C sensors share one bus with the board’s relay expander and clock/RTC; the temperature probe uses a separate 1-Wire pin.

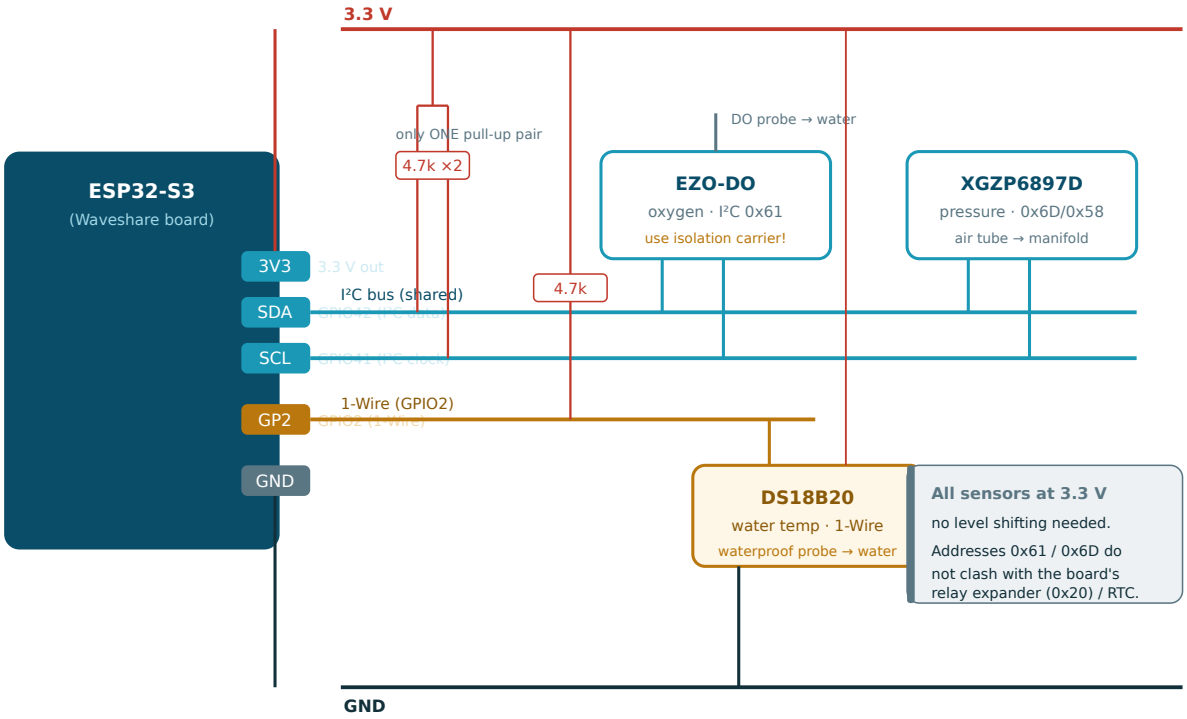

Figure 3: Sensor wiring: the oxygen (`0x61`) and pressure (`0x6D`/`0x58`) sensors share the I²C bus (`SDA`=GPIO42, `SCL`=GPIO41) with a **single** 4.7 kΩ pull-up pair; the DS18B20 sits on a 1-Wire GPIO with its own 4.7 kΩ pull-up. The oxygen probe and the temperature probe go into the water; the pressure sensor gets a short air tube.

1. Power every sensor from the board's **3.3 V** and **GND**.
2. Connect the two I²C sensors' `SDA`/`SCL` to `GPIO42`/`GPIO41`. Fit **only one** 4.7 kΩ pull-up pair for the whole bus — if a sensor breakout already has pull-ups, remove them.
3. Wire the DS18B20 data line to a free GPIO (e.g. `GPIO2`) with a 4.7 kΩ pull-up to 3.3 V.
4. For oxygen, put the EZO-DO on an **electrically isolated carrier** [5] and keep it inside the enclosure — only the probe cable goes to the water.

> **TIP** **Keep I²C short**
>
> I²C only reaches ≈1–3 m. Mount the oxygen and pressure circuits **in the controller enclosure** and run the **probe/tube** outside. The DS18B20 (1-Wire) is the one sensor made for a long run into the water.

9.5 Using the ESP32 backend

With the board flashed and wired, the adapter can drive it directly — no relay adapter in between.

1. **Flash the firmware in your browser** — no PlatformIO, no command line: open the flash page in **Chrome** or **Edge** on a computer, connect the ESP over USB, click **Install firmware**, pick the serial port and confirm. After about a minute the board reboots running the firmware.
2. **Power & wire** per the failsafe diagram (valves 24 V DC on NC, emergency valve NO, the 230 V AC pump via a relay + snubber or a contactor) — see the **Failsafe wiring** section below.

3. In the adapter's **General** tab set **Backend** = `ESP32 (direct)`, enter the board's **host / IP**, and map the **emergency-valve relay** and **pump relay** (0–7). Each aeration point uses the relay channel set per point.
4. Save. The adapter checks the firmware, pushes the safety configuration, and starts polling.

TIP **Host, port and "Test connection"**

Enter the ESP32's **host / IP** (find it in your router or on the device's own screen). Leave the **port** at **80** — the reference firmware always serves on port 80; a different value only works with a reverse proxy in front. Click **Test connection** to have the running instance contact the device: it confirms host and port are right and shows the firmware version. Note: the per-point mapping follows the chosen backend — **ESP32 relay channels appear only when the backend is `ESP32 (direct)`**; with the ioBroker-state backend every point maps to an ioBroker state.

SAFETY **The failsafe lives on the device**

The adapter sends a **heartbeat**; if it stops (network or ioBroker down) the firmware protects the pond on its own — it opens the emergency valve and switches the pump off, and it enforces the dead-head interlock locally. This is why the ESP32 is driven directly rather than switched "dumb" from afar.

TIP **Autonomous schedule (optional)**

Turn on **Autonomous schedule (run without ioBroker)** in the General tab and the adapter also pushes your **time schedules** to the device. Now, if the heartbeat is lost, the ESP32 does not just fail safe — it **keeps running your schedule** against its own NTP clock, so the pond stays aerated while ioBroker or the network is down. The pump only runs while at least **minimum open valves** are open (otherwise pump off + emergency valve open), and the dead-head interlock still overrides everything. The cyclic round-robin **sequence** is not run autonomously; it stays with the adapter.

TIP **Which firmware belongs to which adapter**

The adapter and the firmware are matched by a **protocol version** — the hard contract — not by exact release numbers, so a firmware bug-fix does not force a new adapter. Each adapter version also names a **recommended** and **minimum** firmware for convenience:

Adapter version	Firmware
0.0.20 +	protocol 1 · recommended v1.1.0 · minimum v1.0.0

On connect the adapter reads the device's version and publishes it as `info.deviceFirmware` with a `info.firmwareCompatible` flag; a protocol mismatch is logged as an error and an outdated firmware as a warning. The ESP32 configuration tab shows the recommended version and links to the firmware releases.

TIP **Licensing & activation (optional)**

If your firmware ships the optional **licensing overlay**, the device runs one of three tiers: **free** (monitoring only), **community** (relay control) or **pro** (+ the autonomous standalone schedule). Safety always runs regardless of tier — failsafe, emergency valve, dead-head interlock and the hand buttons are never locked. A brand-new device runs fully (**pro**) for a **30-day trial**, then falls back to free until you enter an activation key on the device's `/license` page (which shows the **device code** you give when unlocking). The adapter mirrors the status into `info.licenseTier`, `info.licenseTrialDaysLeft` and `info.deviceCode`; if the device is **not licensed for control**, monitoring keeps working and control commands are skipped with one clear hint (see `info.licenseControlBlocked`) instead of repeated errors. Public firmware built without the overlay is unaffected — control stays open.

Custom names (from tier community): you can give the aeration relay channels (Ch 1–6) and the button inputs (DI 1–8) friendly names, shown on the device web UI. Set them either in the **adapter** (the aeration-point names and an optional per-point button name are pushed to the device) or — in standalone operation without an adapter — on the device under **Settings → Names (channels & buttons)** (stored in NVS). **Ch 7 = emergency valve** and **Ch 8 = pump** are fixed. Without a licence the pages show the default `Ch 1…8` / `DI 1…8` labels.

SAFETY **Re-enter the activation key after re-flashing**

The activation key is **stored on the ESP** (in the NVS area). **Re-flashing through the installer / flash page** overwrites that area and **erases the stored key** — the device then restarts into the 30-day trial. This is intentional (a clean first flash) and **nothing to worry about**: the **device code is immutable** because it is derived from the hardware (factory MAC), not from storage. So the **same activation key keeps working** — open the `/license` page and paste the **same key** again; a new key is not required. **Keep the activation key from the e-mail somewhere safe.**

Important difference: a normal **firmware update via the device's "Update" page** (the *Install update online* button or a file upload) only writes the app partition and leaves NVS untouched — **the activation and all settings are kept**. So for later updates use the Update page, not the installer.

TIP **On-device web UI (port 80)**

Open the board's IP in a browser — the firmware serves seven self-contained pages (no cloud, no app): **Home** (watch the relays / sensors and toggle channels on site), **Schedule** (view / edit the autonomous schedule on the device), **Settings** (DHCP or static IP / DNS / hostname, the **NTP server** — default `de.pool.ntp.org` —, the WS2812 LED / buzzer and the licensed channel/button **names**), **Licence** (read the device code and enter an activation key), **Update** (an **over-the-air** firmware update: automatic version check and a **one-click online update** straight to the device — activation and settings are kept —, or a file upload, plus a **Restart** button), **Info** (time, IP, MAC, hostname, version, memory and uptime) and **Log** (a live diagnostic log — boot, Ethernet, licence, mode and OTA update with the exact failure reason — for analysing problems over the network). Time is kept by NTP and backed by the on-board **RTC**, so the clock survives a power loss and NTP outages. The status LED shows green = normal, orange = no link, blue = a button override is active, red-blinking = failsafe; the buzzer beeps once when the failsafe engages. A step-by-step beginner install & activation guide (English + German) is the flash page itself.

TIP **WiFi (optional)**

The board’s ESP32-S3 also has native 2.4 GHz **WiFi**, usable as an alternative or in addition to the W5500 Ethernet. Enable it and enter **SSID + password** on the **Settings** page (the password is never read back). Enabling, disabling or switching networks applies **live** — no reboot. Ethernet stays the default route while its cable is plugged in, so a wired install is unaffected; unplug the cable and the device keeps running over WiFi. Handy in the garden by the pond where a LAN cable is not always at hand.

SAFETY **WiFi needs the external antenna — mandatory**

WiFi works **only with the external 2.4 GHz antenna** fitted to the module’s antenna connector — the “1U” module has **no** on-board antenna. The reference device **ships with this antenna**; connect it before enabling WiFi. Ethernet / PoE does not need it.

TIP **First setup without LAN — the setup hotspot**

At the pond a network cable is not always at hand. For that case the firmware includes a **setup hotspot**: if neither LAN nor WiFi comes up within **25 seconds** of boot, the device opens its own WiFi **“pond-aeration-setup”** (password **“pondsetup”**) with a *captive portal*.

1. Join that WiFi from a phone — the config page usually opens by itself (otherwise open `http://192.168.4.1/`) and you land on **Settings**.
2. Under **WiFi** enter your home network (SSID + password) and **Save**. The device adopts your network and **closes the hotspot automatically** once it has an IP there.
3. Reconnect your phone to your normal WiFi and open the device there (hostname **pond-aeration**, or as shown by your router).

The hotspot can also be opened/closed **deliberately** with a button on the **Settings** page (e.g. to move the device to a different WiFi later); the automatic fallback can be turned off there too. The setup hotspot **also requires the external antenna**.

10 FAQ

Do I need an ESP32 to use the adapter? No. By default it controls valves and the pump through existing ioBroker states, so any relay board works. The direct ESP32 build is an optional convenience (no extra PC, an on-device failsafe and a mobile web page).

Nothing switches — did I break something? Check that the **master switch** is on, the **mode** is `auto` (or `manual` with a point opened), and each enabled point has a **valve state** mapped. In **dry-run** nothing is switched by design.

Can I run just a winter ice-free hole? Yes. Enable **Winter / ice-free mode** with your season window and (optionally) frost protection, and leave the automatic schedule empty.

Is oxygen monitoring required? No, it is fully optional — and the most expensive, most maintenance-heavy part. Many ponds run fine on a schedule plus the safety interlock.

Will it keep my fish safe on its own? Not yet — see the warning on the cover. It is still in development. Always supervise it and keep an independent failsafe.

11 Troubleshooting

The admin page is empty / old	After updating, run `iobroker upload automatic-pond-aeration` and reload the browser with Ctrl+F5. The admin files are cached.
Address search says “no address found”	The adapter **instance** must be running to answer the lookup, and it must be an up-to-date version. Start the instance and retry.
The safety interlock keeps tripping	The pump is seen as running while too few valves are open. Check the pump state mapping, raise the number of scheduled/open points, or verify the emergency valve wiring.
Oxygen reading looks wrong	Feed the sensor the **water temperature** (it compensates on that), make sure there is **flow** across the probe, and check the membrane/electrolyte. Cheap clones drift badly.
Pressure value jumps around	Humid air can condense on the pressure sensor. Mount it above the manifold with a dead-leg and a moisture trap, barb pointing down [7].

If you are still stuck, open an issue on the project repository [2] with your adapter version, your configuration and the relevant log lines (set the instance log level to `debug`).

12 Glossary

Aeration point One place in the pond that receives air, switched by one valve. Up to 8.

Diffuser / air stone The part under water that turns airflow into fine bubbles.

Solenoid valve An electrically switched valve. “Normally closed (NC)” is shut without power; “normally open (NO)” is open without power.

Dead-heading A pump running against fully closed valves — dangerous. The safety interlock prevents it.

Interlock A safety rule that overrides everything: while the pump runs, at least one valve stays open, or the emergency valve opens and the pump stops.

Make-before-break Opening the next valve before closing the previous one, so there is never a moment with everything shut.

Schedule A weekday time window (`From–To`) during which chosen points/groups run.

Round-robin Rotating through the points, each open for a fixed **dwell** time.

Sequence A round-robin you define step by step, where each step is a point **or** a group, with its own optional dwell — points and groups may be mixed.

Group Several aeration points controlled together. There are never more groups than points.

Dwell time How long one point/step stays open before the cycle moves on.

Dry-run A test mode — the logic runs but no hardware is switched; intended actions are only logged.

Hysteresis A margin that stops an alarm/loop from flickering on and off around a threshold.

Dissolved oxygen (DO) Oxygen in the water, in mg/L, that the animals breathe. “Saturation %” is how full the water is relative to the maximum at that temperature.

State / data point A named value in ioBroker the adapter reads or writes (e.g. a valve switch).

ioBroker The open-source smart-home platform this adapter runs in [1].

ESP32 A small networked microcontroller that can drive the relays and read the sensors directly (the direct ESP32 build).

I²C / 1-Wire Two simple wiring “buses” for sensors — I²C for the oxygen and pressure sensors, 1-Wire for the DS18B20 temperature probe.

13 References

[1] ioBroker — official documentation — https://www.iobroker.net/#en/documentation/

[2] ioBroker.automatic-pond-aeration — project repository — https://github.com/ssbingo/ioBroker.automatic-pond-aeration

[3] Waveshare ESP32-S3-POE-ETH-8DI-8RO — product wiki — https://www.waveshare.com/wiki/ESP32-S3-POE-ETH-8DI-8RO

[4] Atlas Scientific EZO-DO — dissolved-oxygen circuit datasheet — https://files.atlas-scientific.com/DO_EZO_Datasheet.pdf

[5] Atlas Scientific — Electrically Isolated EZO Carrier Board — https://atlas-scientific.com/carrier-boards/electrically-isolated-ezo-carrier-board-gen-2/

[6] DFRobot Gravity — Analog Dissolved Oxygen Sensor (SEN0237-A) — https://wiki.dfrobot.com/sen0237-a/

[7] CFSensor XGZP6897D — I²C pressure sensor — https://cfsensor.com/product/xgzp6897d/

[8] fanfanlatulipe26/XGZP6897D — Arduino library — https://github.com/fanfanlatulipe26/XGZP6897D

[9] ESPHome — xgzp68xx sensor component — https://esphome.io/components/sensor/xgzp68xx/

[10] Analog Devices — DS18B20 datasheet — https://www.analog.com/media/en/technical-documentation/data-sheets/ds18b20.pdf

[11] Maxim AN148 — reliable long-line 1-Wire networks — https://www.analog.com/en/resources/technical-articles/guidelines-for-reliable-long-line-1wire-networks.html

[12] cpetrich/counterfeit_DS18B20 — how to spot fakes — https://github.com/cpetrich/counterfeit_DS18B20

[13] ioBroker.automatic-feeder — the coupled feeder adapter — https://github.com/ssbingo/ioBroker.automatic-feeder

[14] SunCalc — sun position / times library — https://github.com/mourner/suncalc

[15] OpenStreetMap Nominatim — geocoding usage policy — https://operations.osmfoundation.org/policies/nominatim/

[16] BerryBase — DS18B20 waterproof probe (reputable EU source) — https://www.berrybase.de/ds18b20-ic-digitaler-temperatursensor-wasserdicht